UNIVERSITÉ LAVAL

Solutionnaire du TP1

À votre tour

0. Importer et compter

```
res <- read_tsv("/home/public/enrichissement/airway_deseq2.tsv")
```

```
nrow(res)
#> [1] 22369
testes <- res |> filter(!is.na(padj))
nrow(testes)
#> [1] 18032
de <- testes |> filter(padj < 0.05, abs(log2FoldChange) > 1)
nrow(de)
#> [1] 1018
dplyr::count(de, sens = if_else(log2FoldChange > 0, "induit", "réprimé"))
#> # A tibble: 2 × 2
#>   sens        n
#>   <chr>   <int>
#> 1 induit    531
#> 2 réprimé   487
```

`filter()` écarte les lignes où la condition vaut `NA` : les gènes non testés ne sont pas comptés. Avec un crochet (`res[res$padj < 0.05, ]`), ils le seraient.

1. Explorer la base et sa version

```
keytypes(org.Hs.eg.db)
#>  [1] "ACCNUM"       "ALIAS"        "ENSEMBL"      "ENSEMBLPROT"  "ENSEMBLTRANS"
#>  [6] "ENTREZID"     "ENZYME"       "EVIDENCE"     "EVIDENCEALL"  "GENENAME"
#> [11] "GENETYPE"     "GO"           "GOALL"        "IPI"          "MAP"
#> [16] "OMIM"         "ONTOLOGY"     "ONTOLOGYALL"  "PATH"         "PFAM"
#> [21] "PMID"         "PROSITE"      "REFSEQ"       "SYMBOL"       "UCSCKG"
#> [26] "UNIPROT"
md <- AnnotationDbi::metadata(org.Hs.eg.db)
md |> filter(str_detect(name, "SOURCEDATE"))
#>             name                  value
#> 1   EGSOURCEDATE              2025-Sep24
```

```
#> 2   GOSOURCEDATE               2025-07-22
#> 3 GOEGSOURCEDATE               2025-Sep24
#> 4 KEGGSOURCEDATE               2011-Mar15
#> 5   GPSOURCEDATE                UTC-Sep26
#> 6   ENSOURCEDATE               2025-Sep03
#> 7   UPSOURCEDATE Sat Oct  4 09:48:11 2025
```

Chaque source a sa date. KEGG date de **mars 2011** : la licence de KEGG a changé cette année-là et ses données ne sont plus redistribuées dans ce package. Pour KEGG à jour, il faut passer par son site (`clusterProfiler::enrichKEGG()` l'interroge en ligne). Notez ces dates dans vos méthodes : une autre version donnera d'autres résultats.

2. *BRCA1*

```
AnnotationDbi::select(org.Hs.eg.db, keys = "BRCA1", keytype = "SYMBOL",
                      columns = c("ENTREZID", "ENSEMBL", "GENENAME", "MAP"))
#>   SYMBOL ENTREZID         ENSEMBL                    GENENAME      MAP
#> 1  BRCA1      672 ENSG00000012048 BRCA1 DNA repair associated 17q21.31
uniprot_brca1 <- mapIds(org.Hs.eg.db, keys = "BRCA1", keytype = "SYMBOL",
                        column = "UNIPROT", multiVals = "list")
length(uniprot_brca1$BRCA1)
#> [1] 45
"P38398" %in% uniprot_brca1$BRCA1
#> [1] TRUE
```

Un gène, 45 accessions UniProt : l'entrée revue **Swiss-Prot** (`P38398`) et de nombreuses entrées **TrEMBL** (isoformes, fragments, séquences non revues).

3. Entrez → symbole et Ensembl

```
AnnotationDbi::select(org.Hs.eg.db, keys = c("672", "5468", "7157"), keytype = "ENTREZID"
                      columns = c("SYMBOL", "ENSEMBL", "GENENAME"))
#>   ENTREZID SYMBOL         ENSEMBL
#> 1      672  BRCA1 ENSG00000012048
#> 2     5468  PPARG ENSG00000132170
#> 3     7157   TP53 ENSG00000141510
#>                                       GENENAME
#> 1                      BRCA1 DNA repair associated
#> 2 peroxisome proliferator activated receptor gamma
#> 3                               tumor protein p53
```

BRCA1, *PPARG* et *TP53*. Les clés sont des **chaînes** : les identifiants Entrez sont des nombres, mais `org.Hs.eg.db` les stocke comme du texte.

4. Ajouter le symbole aux gènes DE

```
de_annot <- de |>
  mutate(symbole = mapIds(org.Hs.eg.db, keys = gene, keytype = "ENSEMBL", column = "SYMBO
sum(is.na(de_annot$symbole))
#> [1] 82
de_annot |> arrange(desc(log2FoldChange)) |> head(10)
#> # A tibble: 10 × 7
#>    gene            baseMean log2FoldChange  stat   pvalue     padj symbole
#>    <chr>              <dbl>          <dbl> <dbl>    <dbl>    <dbl> <chr>
#>  1 ENSG00000179593    67.2            9.51  9.02 1.85e-19 1.18e-17 ALOX15B
#>  2 ENSG00000109906   385.             7.35 13.7  9.14e-43 2.32e-40 ZBTB16
#>  3 ENSG00000250978    56.3            6.33  9.36 8.33e-21 6.03e-19 LOC101928819
#>  4 ENSG00000132518     5.65           5.88  4.47 7.72e- 6 8.86e- 5 GUCY2D
#>  5 ENSG00000127954   286.             5.21 10.6  4.55e-26 5.06e-24 STEAP4
#>  6 ENSG00000249364     8.84           5.09  4.42 1.01e- 5 1.13e- 4 LINC02997
#>  7 ENSG00000171819   255.             5.08  6.71 1.94e-11 5.18e-10 ANGPTL7
#>  8 ENSG00000100033    55.4            4.89  9.40 5.60e-21 4.09e-19 PRODH
#>  9 ENSG00000260802     6.72           4.88  4.09 4.25e- 5 4.11e- 4 SERTM2
#> 10 ENSG00000168309   160.             4.84 14.6  1.99e-48 6.53e-46 FAM107A
```

`mapIds()` renvoie **une** valeur par identifiant, dans le même ordre : on peut l'ajouter comme colonne sans risque de décalage. 82 gènes DE n'ont pas de symbole dans cette version de la base : ils disparaîtront de toute analyse qui passe par les symboles. Parmi les plus induits, plusieurs ont un `baseMean` faible (ex. *GUCY2D*) : un grand log2FoldChange calculé sur peu de lectures est moins fiable.

5. Conversion avec `select()` : le un-à-plusieurs

```
conv <- AnnotationDbi::select(org.Hs.eg.db, keys = de$gene, keytype = "ENSEMBL",
                              columns = c("SYMBOL", "ENTREZID"))
nrow(conv)
#> [1] 1031
conv |> dplyr::count(ENSEMBL) |> filter(n > 1)
#>            ENSEMBL n
#> 1  ENSG00000105889 2
#> 2  ENSG00000126861 2
#> 3  ENSG00000137962 2
#> 4  ENSG00000144063 2
#> 5  ENSG00000160200 2
#> 6  ENSG00000160223 2
#> 7  ENSG00000178734 2
#> 8  ENSG00000197943 2
#> 9  ENSG00000213160 2
#> 10 ENSG00000230417 3
#> 11 ENSG00000231924 2
#> 12 ENSG00000259426 2
```

1031 lignes pour 1018 gènes : certains identifiants Ensembl correspondent à **plusieurs** gènes Entrez (le message `'select()' returned 1:many mapping` le signale). Il faut décider quoi en faire (garder le premier, les garder tous, les écarter) et le dire.

6. Interroger GO

```
terme <- AnnotationDbi::select(GO.db, keys = "GO:0001838", keytype = "GOID",
                               columns = c("TERM", "ONTOLOGY", "DEFINITION"))
terme$TERM
#> [1] "embryonic epithelial tube formation"
terme$ONTOLOGY
#> [1] "BP"
terme$DEFINITION
#> [1] "The morphogenesis of an embryonic epithelium into a tube-shaped structure."

parents <- as.list(GOBPPARENTS["GO:0051384"])[[1]]
parents
#>          isa
#> "GO:0031960"
AnnotationDbi::select(GO.db, keys = parents, keytype = "GOID", columns = "TERM")
#>         GOID                        TERM
#> 1 GO:0031960 response to corticosteroid
```

Le nom de chaque élément de `parents` est la **relation** (`isa` : « est un »).

7. Les annotations GO de *FKBP5*

```
go_fkbp5 <- AnnotationDbi::select(org.Hs.eg.db, keys = "FKBP5", keytype = "SYMBOL",
                                  columns = c("GO", "ONTOLOGY", "EVIDENCE"))
nrow(go_fkbp5)
#> [1] 26
dplyr::count(go_fkbp5, ONTOLOGY)
#>   ONTOLOGY  n
#> 1       BP  8
#> 2       CC  8
#> 3       MF 10
dplyr::count(go_fkbp5, EVIDENCE, sort = TRUE)
#>   EVIDENCE n
#> 1      IEA 9
#> 2      IDA 6
#> 3      TAS 3
#> 4      HDA 2
#> 5      IBA 2
#> 6      IPI 2
#> 7       IC 1
#> 8      IMP 1
```

Chaque ligne est une annotation gène → terme, avec son **code d'évidence**. `IEA` (annotation électronique) n'a été vérifiée par personne ; `IDA`, `IMP`, `IPI` reposent sur une expérience publiée.

Et si… ?

1. Des identifiants versionnés

```
mapIds(org.Hs.eg.db, keys = paste0(de$gene, ".1"), keytype = "ENSEMBL", column = "SYMBOL"
#> Error in .testForValidKeys(x, keys, keytype, fks): None of the keys entered are valid
```

```
ids_mixtes <- c(de$gene[1:3], paste0(de$gene[4], ".1"))
mapIds(org.Hs.eg.db, keys = ids_mixtes, keytype = "ENSEMBL", column = "SYMBOL")
#>   ENSG00000003402   ENSG00000003987   ENSG00000004799 ENSG00000004846.1
#>           "CFLAR"           "MTMR7"            "PDK4"              NA
```

Si **tous** les identifiants portent une version : une **erreur**, aucun ne correspond. Si **quelques-uns** seulement : des `NA`, **sans erreur**, et ces gènes disparaissent en silence. Toujours retirer la version : `sub("\\..*", "", ids)`.

2. `select()` masqué

```
res |> select(gene, padj)
#> Error: unable to find an inherited method for function 'select' for signature 'x = "sp
```

`select()` désigne maintenant la fonction d'`AnnotationDbi`, qui ne sait pas traiter un tableau. L'erreur est visible ; la solution : `dplyr::select(gene, padj)`.

3. Un ancien symbole

```
AnnotationDbi::select(org.Hs.eg.db, keys = "SEPT2", keytype = "SYMBOL", columns = "ENTREZ
#> Error in .testForValidKeys(x, keys, keytype, fks): None of the keys entered are valid
AnnotationDbi::select(org.Hs.eg.db, keys = "SEPT2", keytype = "ALIAS", columns = "SYMBOL"
#>   ALIAS  SYMBOL
#> 1 SEPT2 SEPTIN2
#> 2 SEPT2 SEPTIN6
```

`SEPT2` n'est plus un symbole officiel : le HGNC a renommé les septines (*SEPTIN2*) en 2020, parce qu'Excel transformait `SEPT2` en date (2 septembre). Comme **alias**, `SEPT2` désigne **deux** gènes actuels : une ancienne liste de gènes ne se convertit pas sans ambiguïté.

4. `multiVals`

```
mapIds(org.Hs.eg.db, keys = "ENSG00000004866", keytype = "ENSEMBL", column = "SYMBOL")
#> ENSG00000004866
#>          "ST7"
mapIds(org.Hs.eg.db, keys = "ENSG00000004866", keytype = "ENSEMBL", column = "SYMBOL",
       multiVals = "list")
#> $ENSG00000004866
#> [1] "ST7"     "ST7-OT3"
```

`"first"` (le défaut) garde *ST7* et écarte *ST7-OT3* **sans le signaler**. Pour un symbole d'affichage, c'est acceptable ; pour une analyse par Entrez, un gène est perdu.

5. `left_join()` avec une table 1:n

```
de_joint <- de |> left_join(conv, by = c("gene" = "ENSEMBL"))
c(avant = nrow(de), apres = nrow(de_joint))
#> avant apres
#>  1018  1031
dplyr::count(de_joint, sens = if_else(log2FoldChange > 0, "induit", "réprimé"))
#> # A tibble: 2 × 2
#>   sens        n
#>   <chr>   <int>
#> 1 induit    537
#> 2 réprimé   494
```

La jointure **duplique** les gènes à correspondance multiple : 1031 lignes au lieu de 1018. Compter les gènes DE (ou les induits) sur ce tableau donnerait un nombre **gonflé**, sans message. Comparer `nrow()` avant et après chaque jointure.